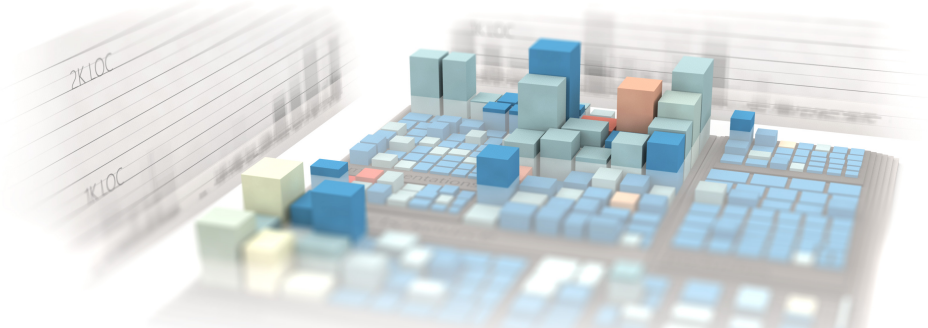
Interactive, Height-based Filtering in 2.5D Treemaps

VINCI 2018 Växjö, Sweden, 13-15 August

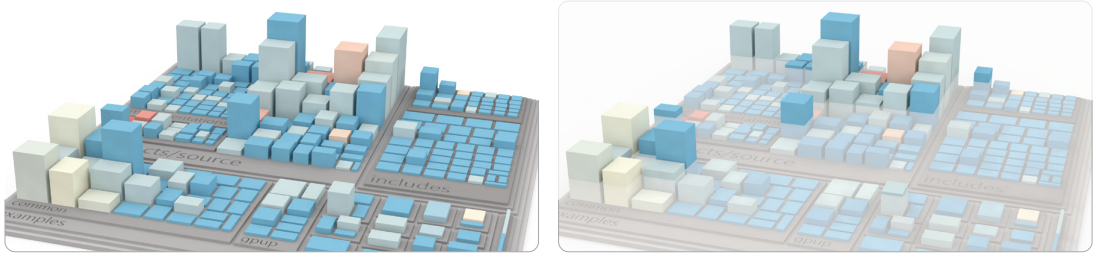


Daniel Limberger, Matthias Trapp, and Jürgen Döllner

Hasso Plattner Institute // Faculty of Digital Engineering // University of Potsdam, Germany

Height Reference

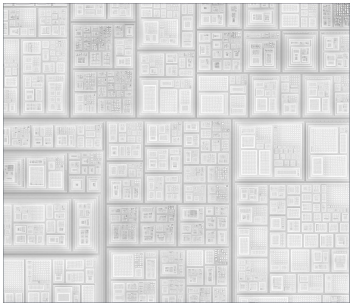
Height reference denotes the visual display of an height threshold that can be interactively modified and used for height based queries in 2.5D treemaps.



Treemaps [3] are used to visualize tree-structured data,

e.g., stock markets [8], sensor data [5], business data [7], file systems [6], software system information [9].

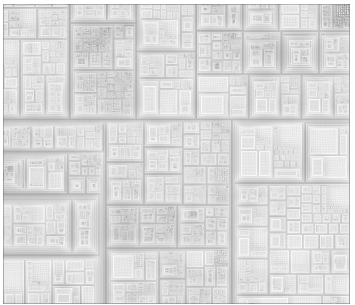
1. Non-spatial data is spatialized—it is given a gestalt that preserves the data's structure, e.g., rectangular treemaps use nested rectangles to depict nodes,
2. and data (attributes) are mapped to visual variables, i.e., properties of the rectangles (size and color).



Treemaps [3] are used to visualize tree-structured data,

e.g., stock markets [8], sensor data [5], business data [7], file systems [6], software system information [9].

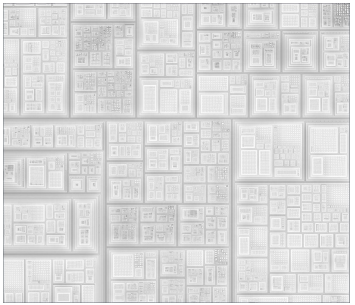
1. Non-spatial data is spatialized—it is given a gestalt that preserves the data's structure, e.g., rectangular treemaps use nested rectangles to depict nodes,
2. and data (attributes) are mapped to visual variables, i.e., properties of the rectangles (size and color).



Treemaps [3] are used to visualize tree-structured data,

e.g., stock markets [8], sensor data [5], business data [7], file systems [6], software system information [9].

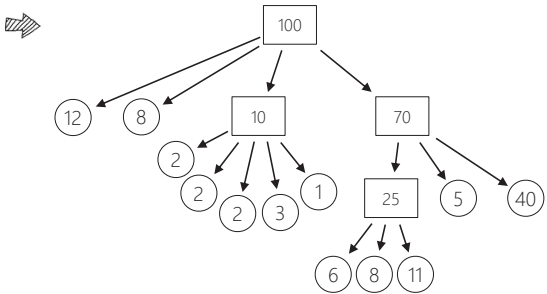
1. Non-spatial data is spatialized—it is given a gestalt that preserves the data's structure, e.g., rectangular treemaps use nested rectangles to depict nodes,
2. and data (attributes) are mapped to visual variables, i.e., properties of the rectangles (size and color).



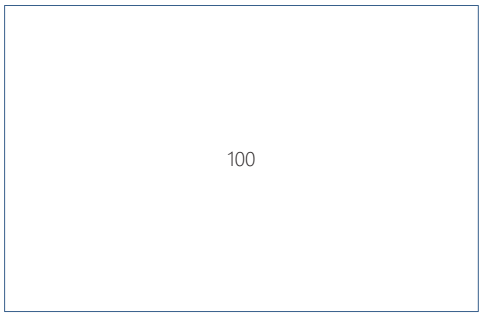
Treemaps — Spatialization of Tree-structured Data

Leaf nodes are colored and have numerical weights, which can represent any associated attribute, e.g., file size. The weight of a parent node is defined by the sum of the weights of its child nodes.

Simple Graph



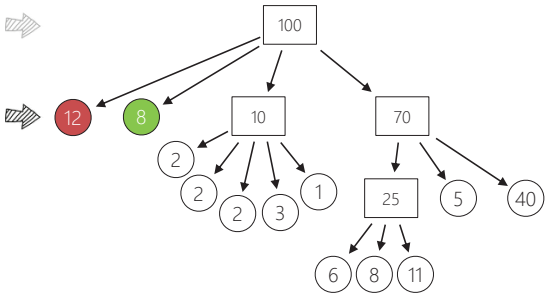
Rectangular Treemap



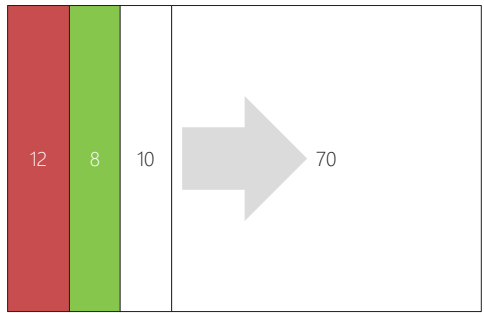
Treemaps — Spatialization of Tree-structured Data

Leaf nodes are colored and have numerical weights, which can represent any associated attribute, e.g., file size. The weight of a parent node is defined by the sum of the weights of its child nodes.

Simple Graph



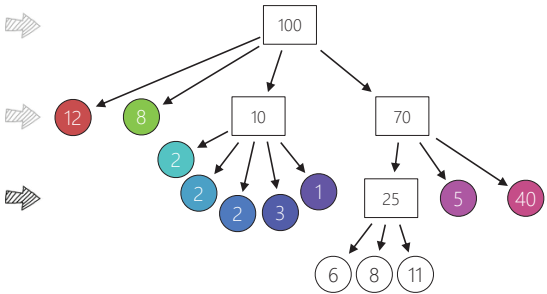
Rectangular Treemap



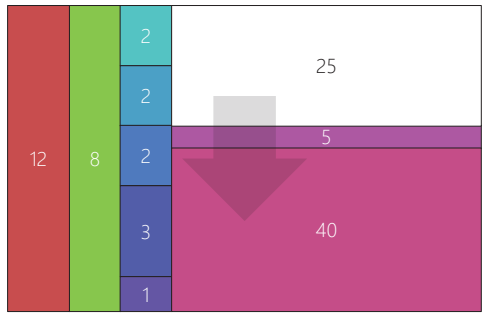
Treemaps — Spatialization of Tree-structured Data

Leaf nodes are colored and have numerical weights, which can represent any associated attribute, e.g., file size. The weight of a parent node is defined by the sum of the weights of its child nodes.

Simple Graph



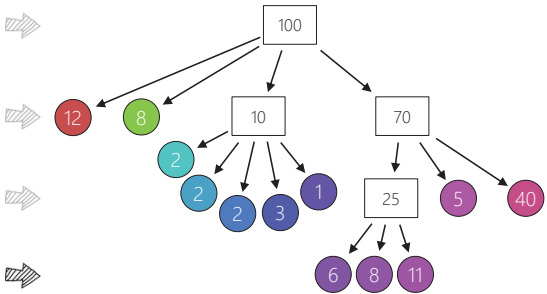
Rectangular Treemap



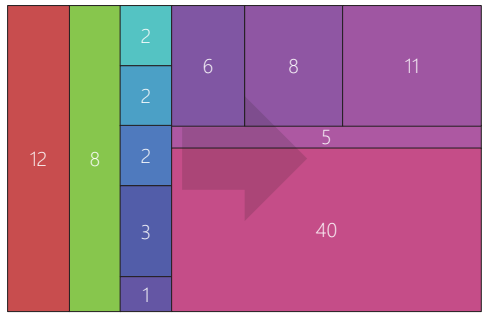
Treemaps — Spatialization of Tree-structured Data

Leaf nodes are colored and have numerical weights, which can represent any associated attribute, e.g., file size. The weight of a parent node is defined by the sum of the weights of its child nodes.

Simple Graph



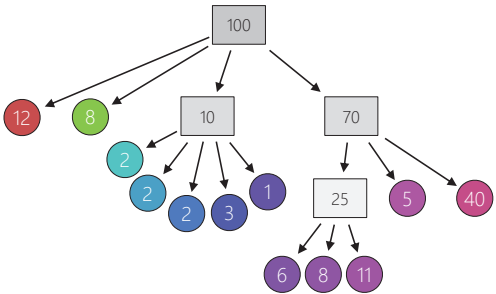
Rectangular Treemap



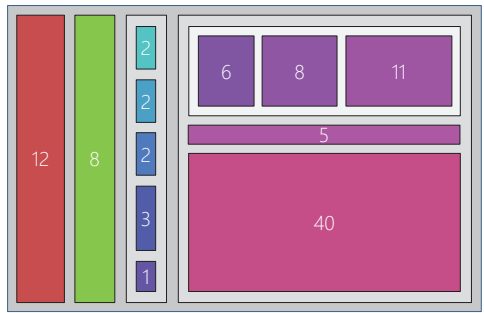
Treemaps — Spatialization of Tree-structured Data

Leaf nodes are colored and have numerical weights, which can represent any associated attribute, e.g., file size. The weight of a parent node is defined by the sum of the weights of its child nodes.

Simple Graph



Rectangular Treemap



Treemaps — Adding Height as Visual Variable

2.5D Treemap [1]

Software Maps — Basic Example

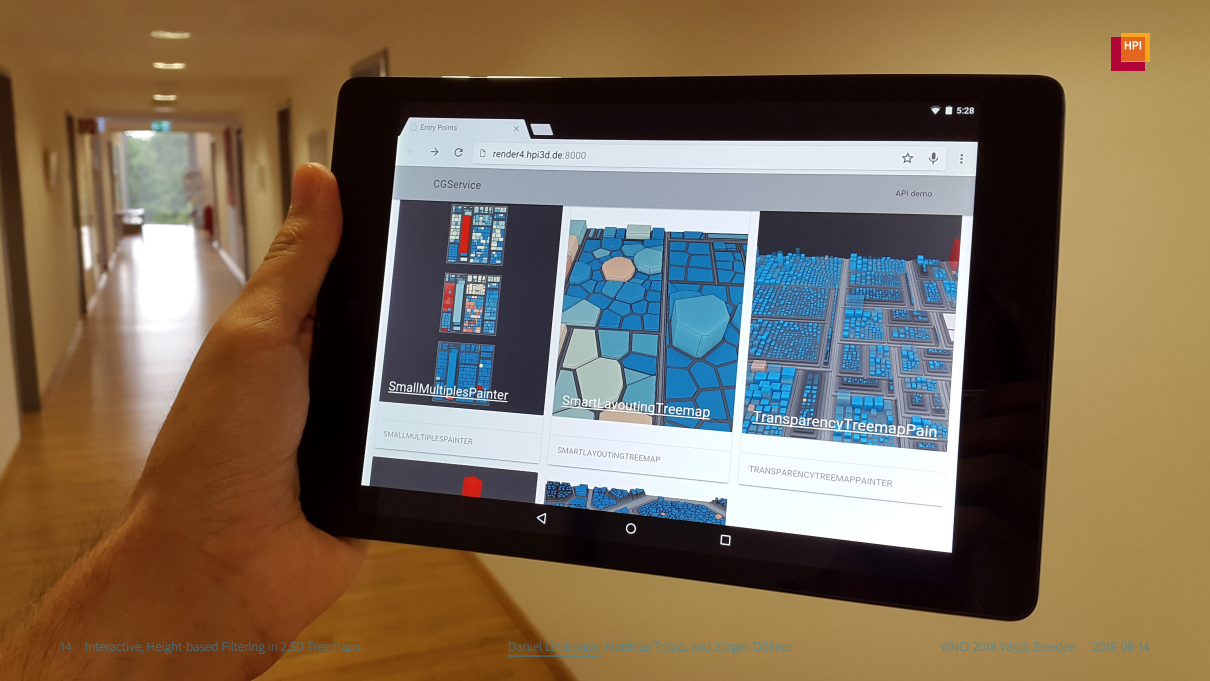
 # complexity
nesting-level or McCabe

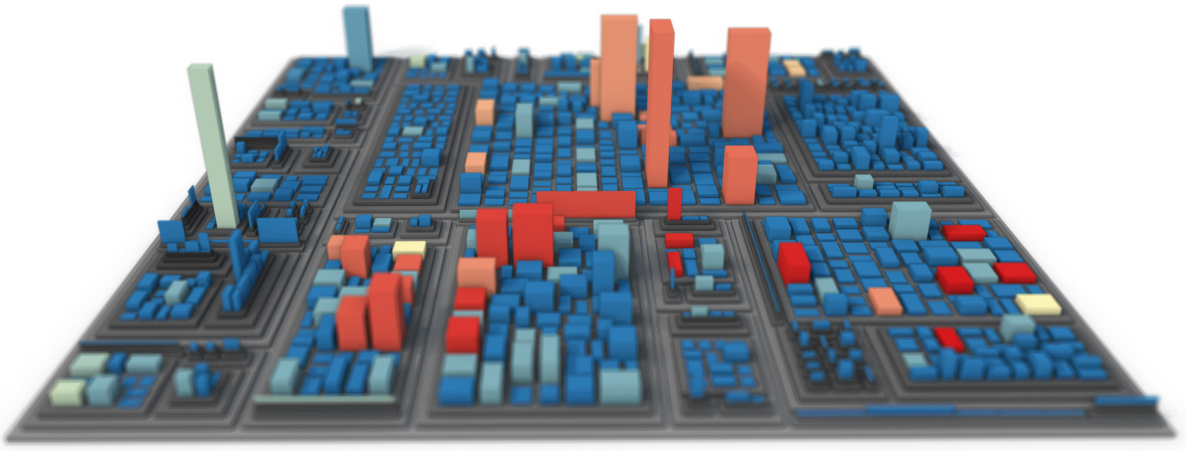
 # developers
that touched this unit

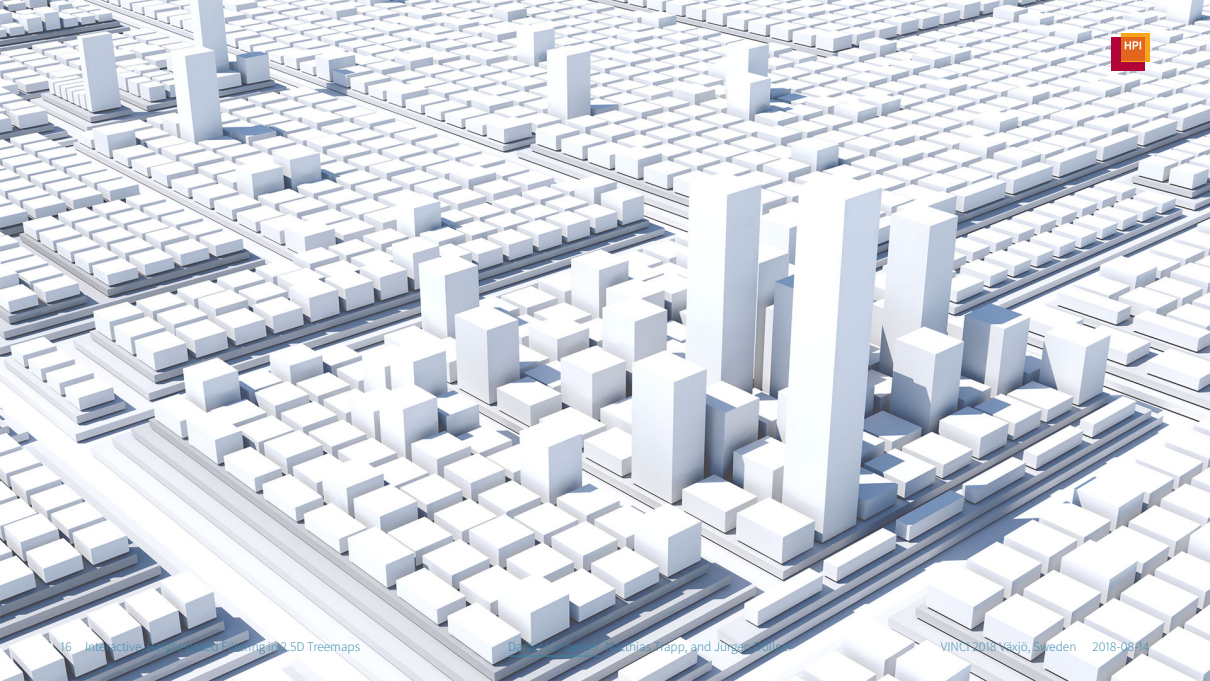
 # lines-of-code

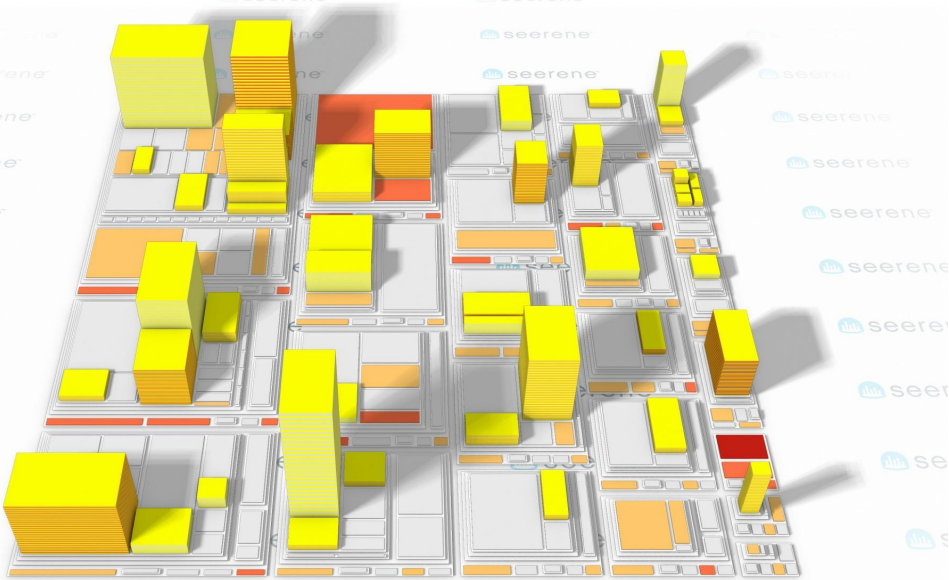
Research Motivation

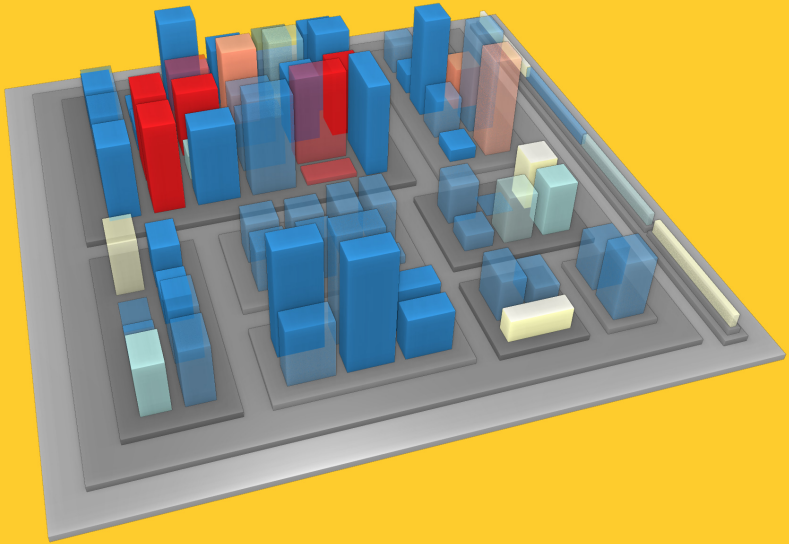
Make 2.5D treemaps a more expressive and interactive communication artifact by increasing its provisioning and overall performance.

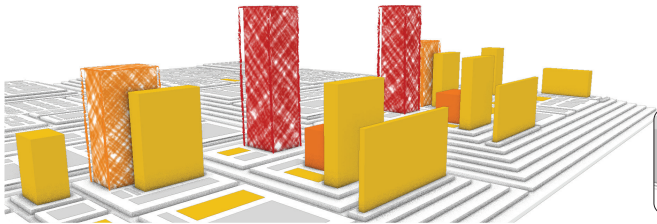

Entry Points
render4.hpi3d.de:8000
CGService
API demo
SmallMultiplesPainter
SMALLMULTIPLESPAINTER
SmartLayoutingTreemap
SMARTLAYOUTINGTREEMAP
TransparencyTreemapPain
TRANSPARENCYTREEMAPPAINTER
5:28

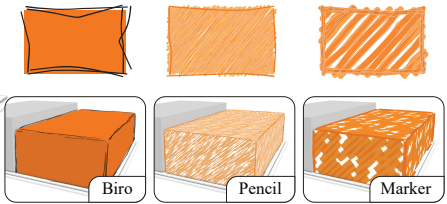


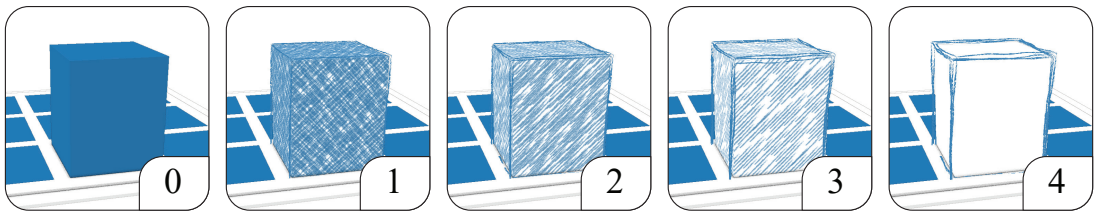


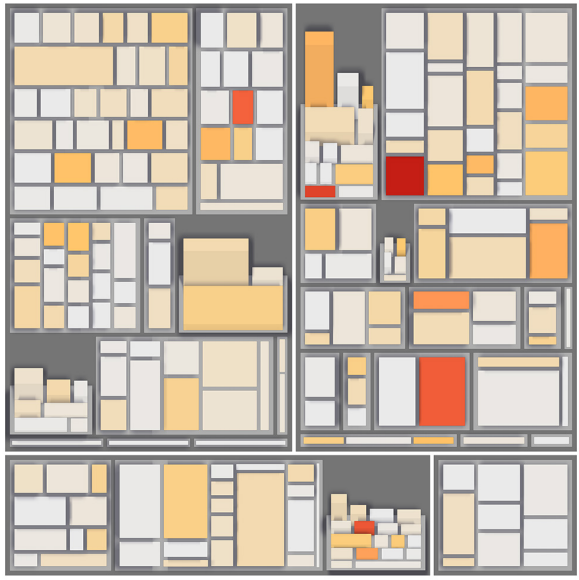


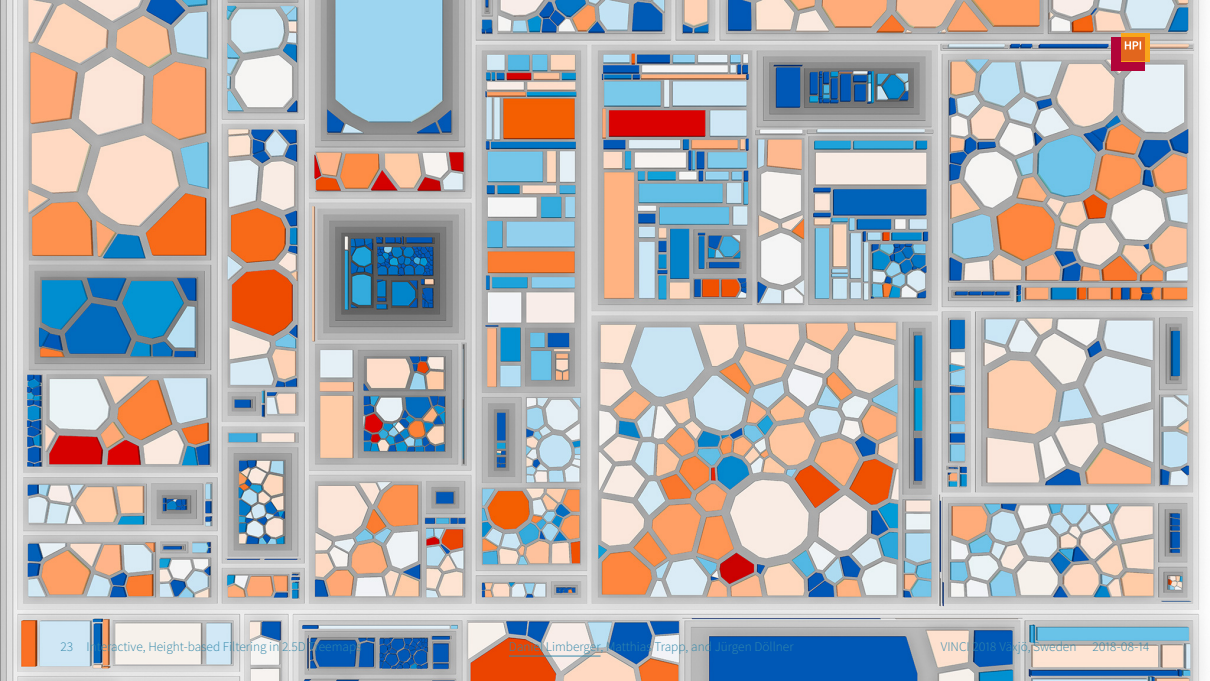




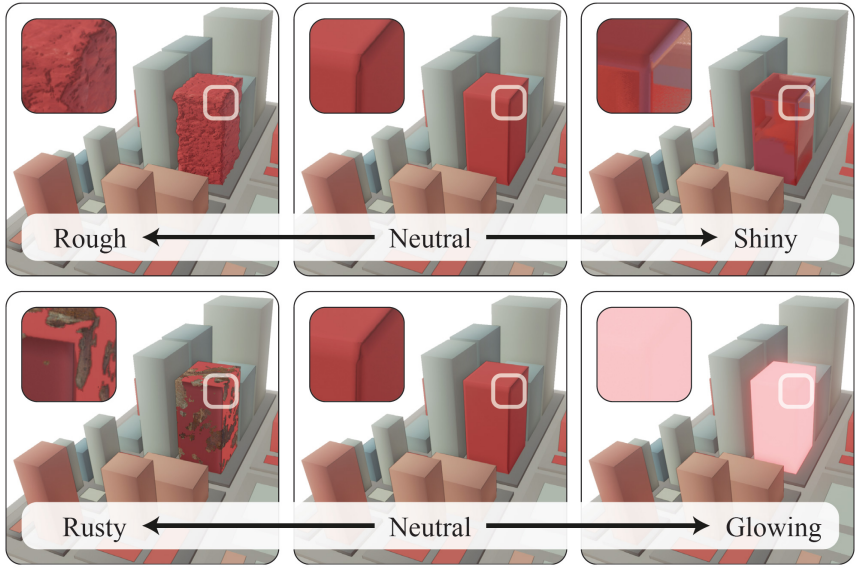

Biro
Pencil
Marker

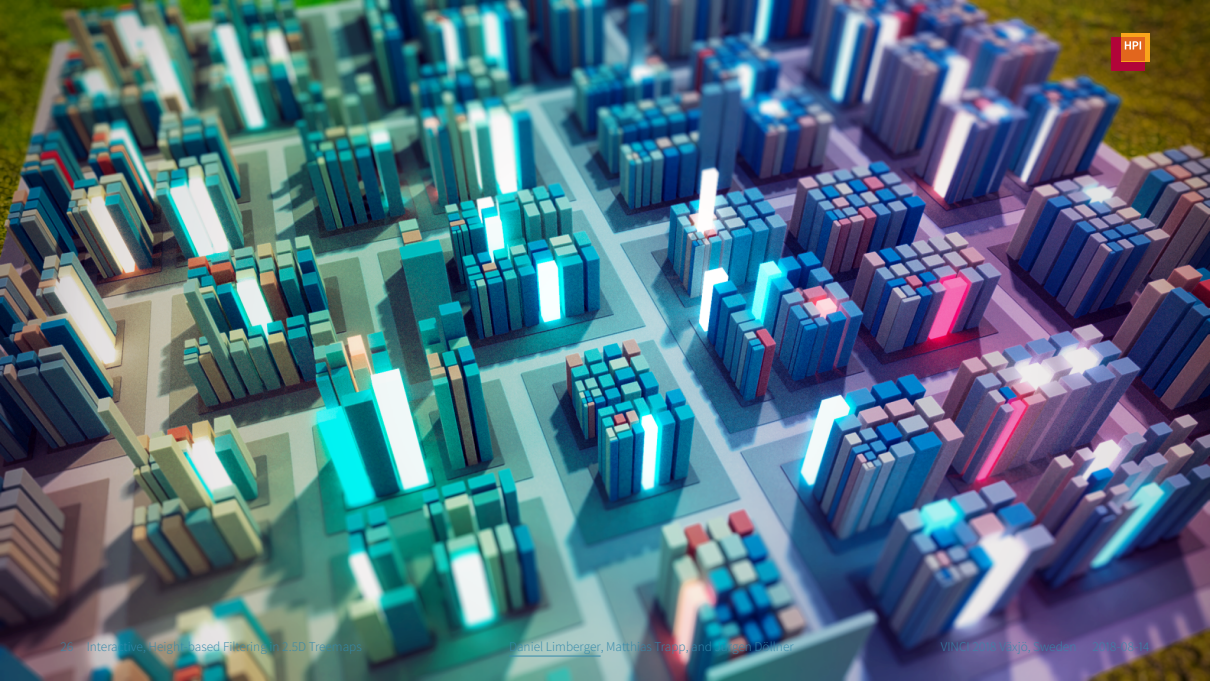

0
1
2
3
4

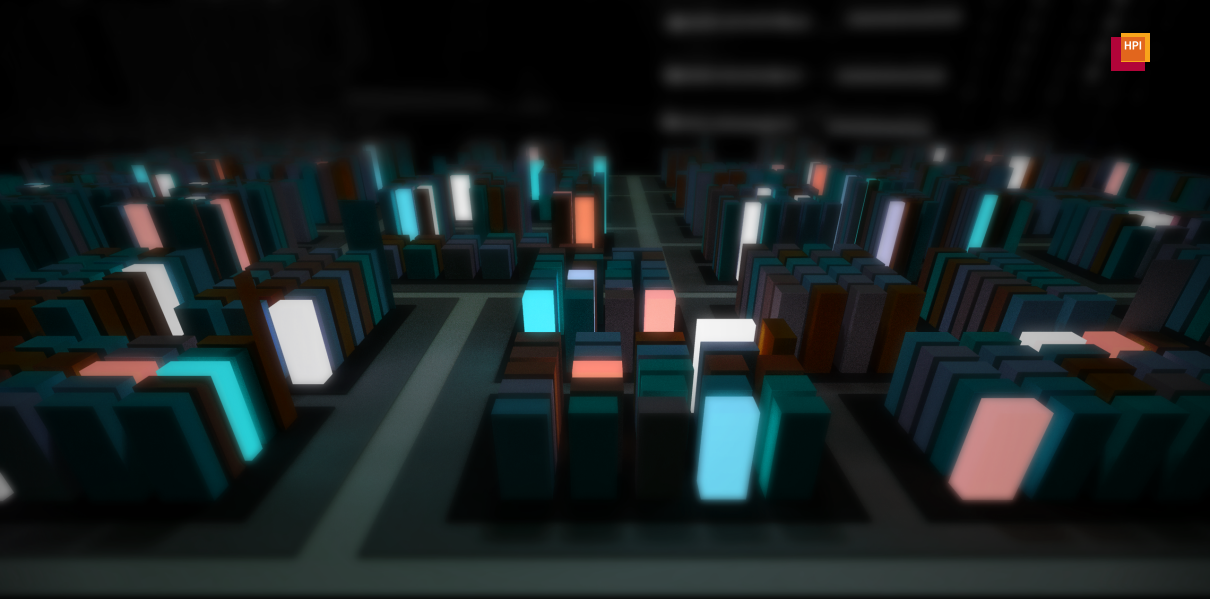


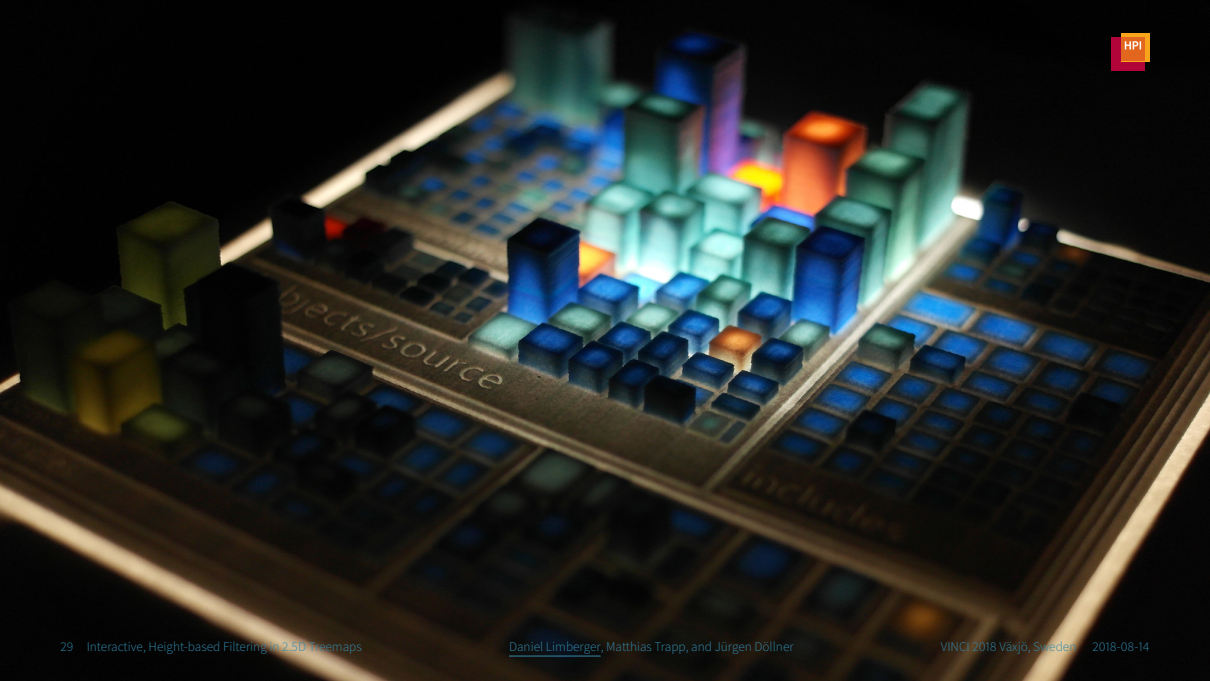

HPI

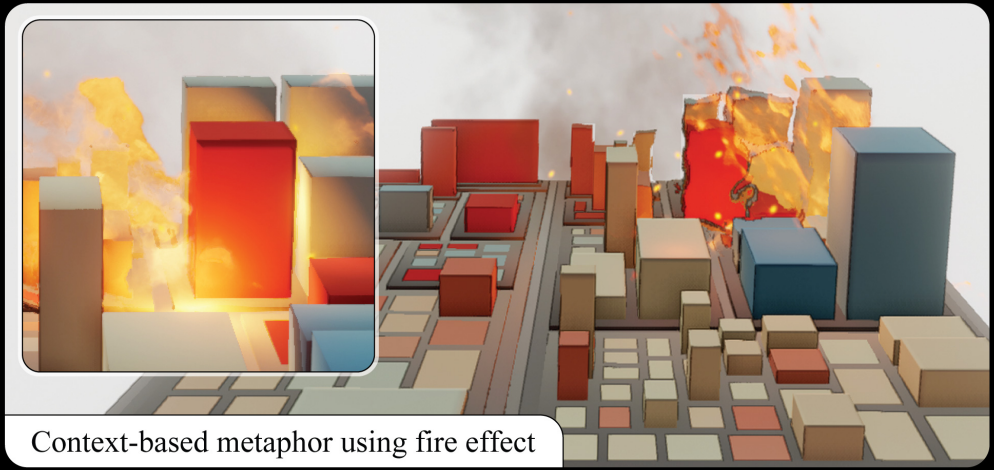
HPI


Rough
Neutral
Shiny
Rusty
Neutral
Glowing

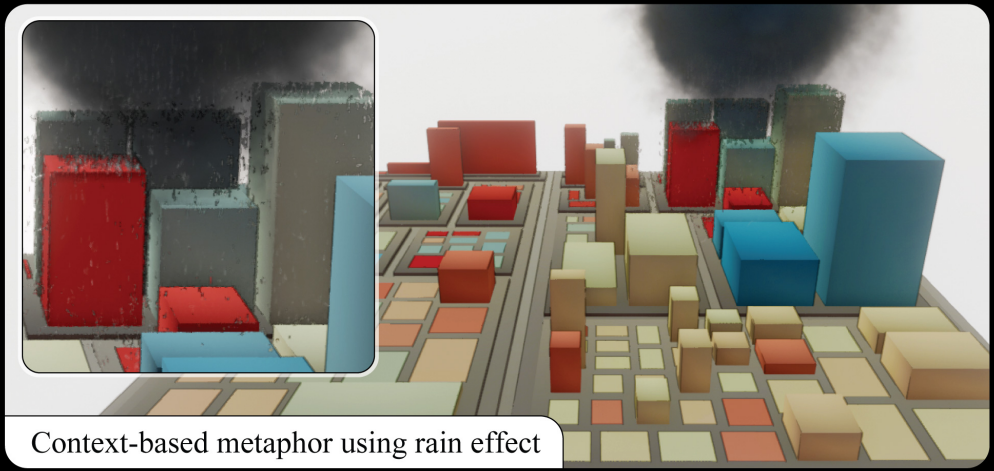


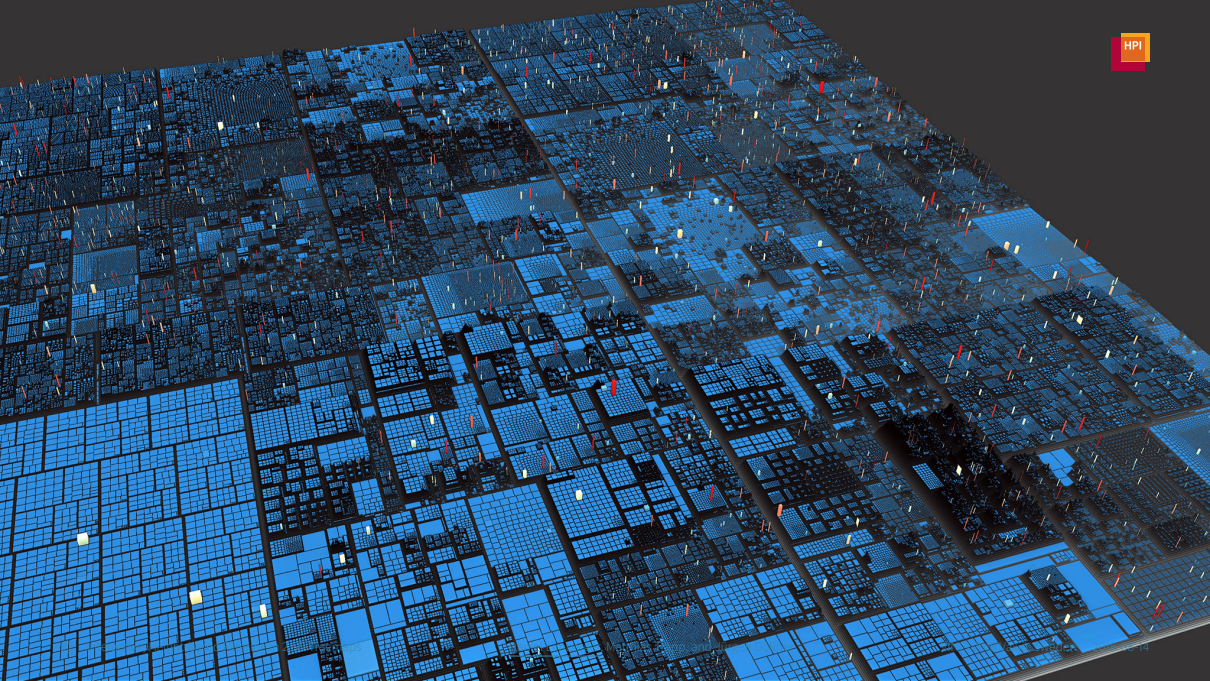


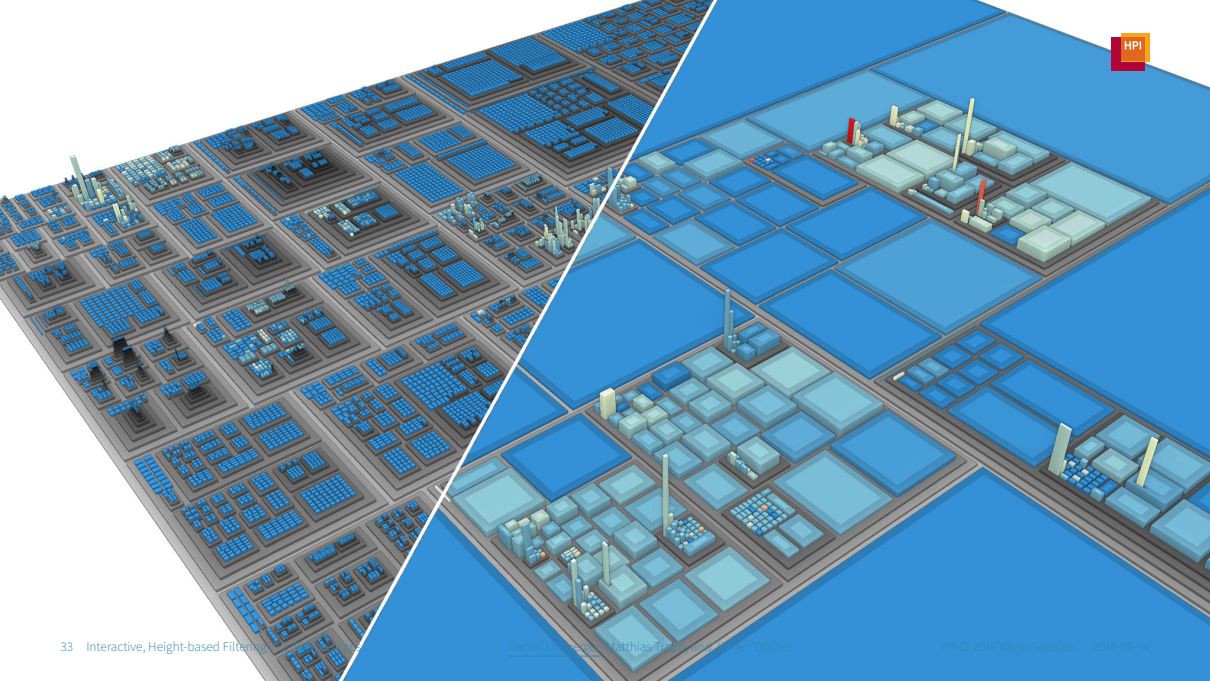

bjects/source



Context-based metaphor using fire effect

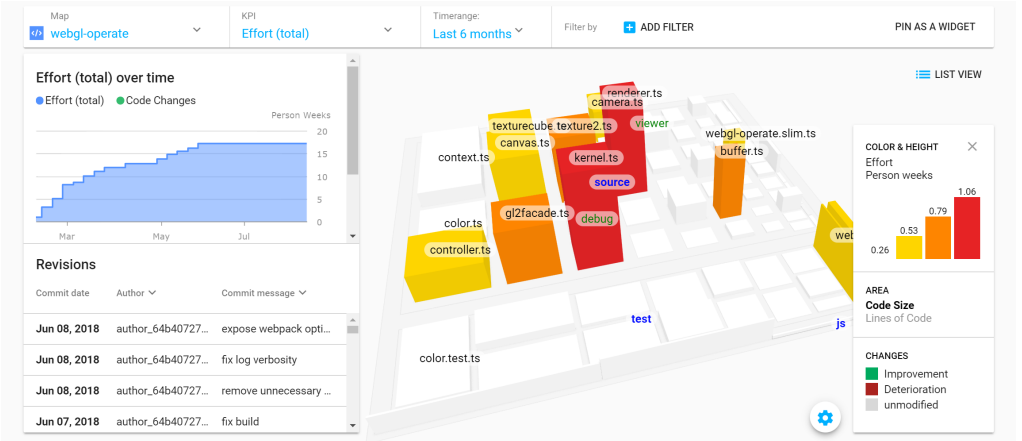

Context-based metaphor using rain effect


HPI



HPI
deploy/scripts
Müller
Schulz
0.448
Schmidt
Meier
webstudio/libs
sd/libs
Meier
0.457
sd/tests
sd/rdf
hpi/tests
Schmidt
0.352
hpi/libs
Schmidt
0.432
hpi/apps
Müller
ogdf
0.335
cga/vis
cga/util
analysis/util

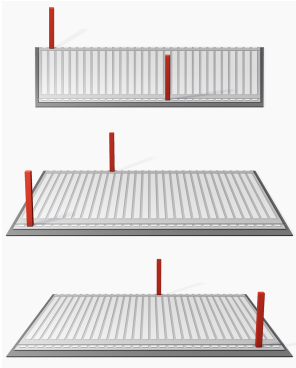
Oberservations

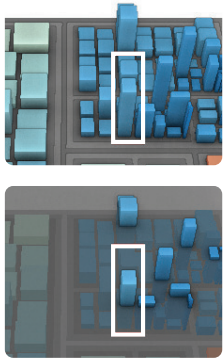


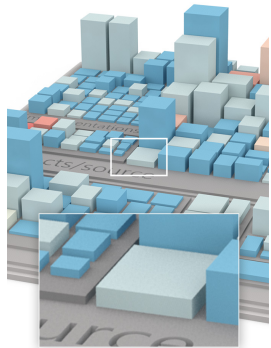
Height is often used merely as rough indication of data trends or color reinforcement.

Problem Statement

- Capability of height-related queries for filtering and identification of nodes-of-interest is missing.
- User height estimates might be hindered due to perspective foreshortening, occlusion, or height-distorting inner node representations:

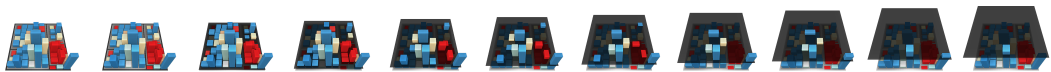






Height Reference

Height reference denotes the visual display of an height threshold that can be interactively modified and used for height-based queries in 2.5D treemaps.

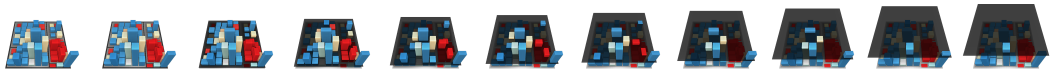


Height-based queries:

- Highest Nodes — Communication of order for identification of n top-most nodes.
- Similar Nodes — Accentuation of nodes of similar or same height (peers).
- Thresholded Nodes — Exclusion or inclusion of nodes below or above a threshold respectively.

Height Reference

Height reference denotes the visual display of an height threshold that can be interactively modified and used for height-based queries in 2.5D treemaps.

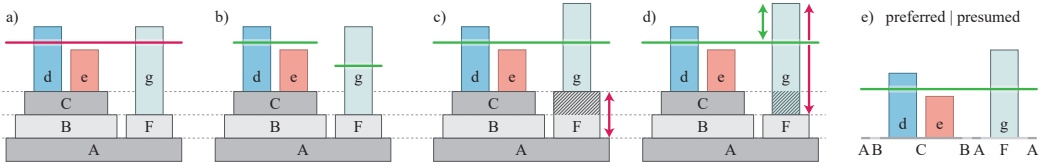


Height-based queries:

- Highest Nodes — Communication of order for identification of n top-most nodes.
- Similar Nodes — Accentuation of nodes of similar or same height (peers).
- Thresholded Nodes — Exclusion or inclusion of nodes below or above a threshold respectively.

Visual Display — Prerequisites

- Simultaneous use of cuboids for leaf and inner nodes is avoided (or applied with care).



Nodes at different heights, single reference height.

Nodes at different heights, per-parent reference heights.

Nodes at the same height, per-parent adjustments.

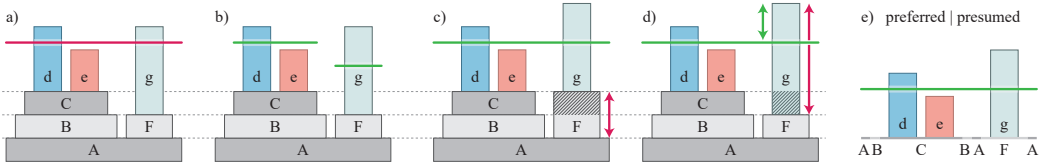
Nodes at different heights, per-node adjustments.

Nodes at same height, no parent heights.

- The mapping correlates cuboid height to relevance for any specific task. If small or negative values are relevant for a specific task, an inverted height mapping is expected (highest node).
- The height range is rather small to avoid occlusion and promote pre-attentive color processing.

Visual Display — Prerequisites

- Simultaneous use of cuboids for leaf and inner nodes is avoided (or applied with care).



Nodes at different heights, single reference height.

Nodes at different heights, per-parent reference heights.

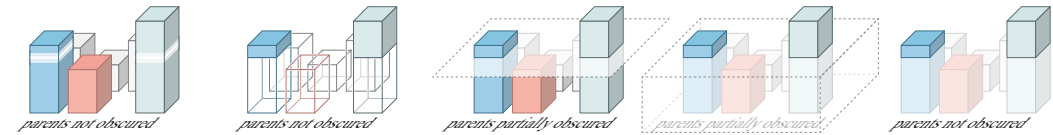
Nodes at the same height, per-parent adjustments.

Nodes at different heights, per-node adjustments.

Nodes at same height, no parent heights.

- The mapping correlates cuboid height to relevance for any specific task. If small or negative values are relevant for a specific task, an inverted height mapping is expected (highest node).
- The height range is rather small to avoid occlusion and promote pre-attentive color processing.

Visual Display — Variations and Constraints

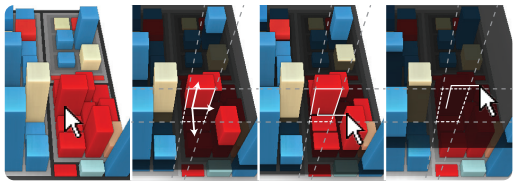


a) Intersection b) Stilts c) Explicit Plane d) Explicit Cuboid e) Implicit Plane

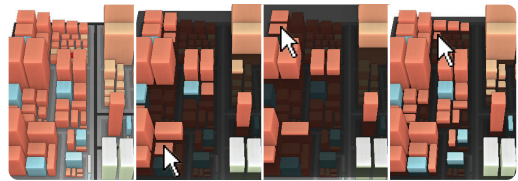
- Should not rely on transparency or translucency (rendering complexity and cost).
- Should be orthogonal to other visual variables (e.g., transparency, procedural texturing, texturing).
- Should not obscure the visual display of inner nodes (e.g., labeling and/or coloring).
- Increasing the height threshold should reduce the overall visual complexity.

Interaction — Approach and Constaints

The height reference can be manipulated by either touching a cuboids lateral faces or its top, and thereby specifying the height threshold (y-coordinate at intersection with cuboid):



Interaction with Lateral Faces

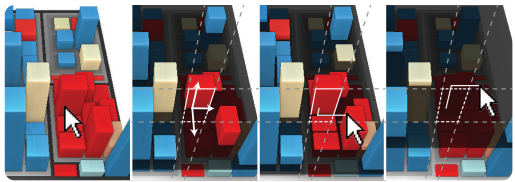


Interaction with Top Face

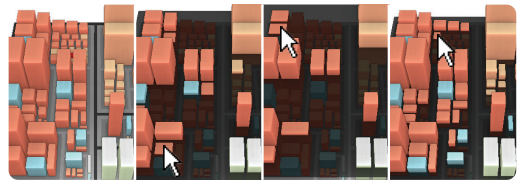
Constraint: 2.5D treemap most likely provides navigation (pan, zoom, rotate), as well as temporary emphasis (e.g., outlining), (un)folding and/or (de)selection of nodes already. Thus, height reference interaction should be exposed using a specific interaction mode (e.g., modifier key).

Interaction — Approach and Constaints

The height reference can be manipulated by either touching a cuboids lateral faces or its top, and thereby specifying the height threshold (y-coordinate at intersection with cuboid):



Interaction with Lateral Faces



Interaction with Top Face

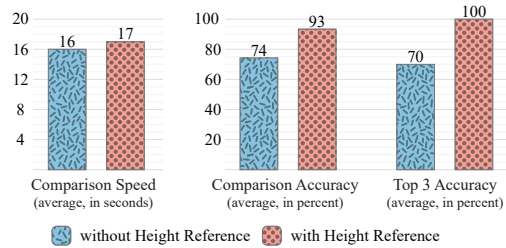
Constraint: 2.5D treemap most likely provides navigation (pan, zoom, rotate), as well as temporary emphasis (e.g., outlining), (un)folding and/or (de)selection of nodes already. Thus, height reference interaction should be exposed using a specific interaction mode (e.g., modifier key).

Preliminary User Study

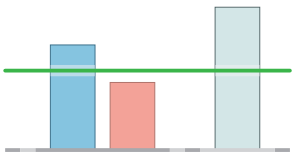
H_1 Users can identify differences in node heights faster when using the height reference.
H_2 Users can identify nodes of specific heights more accurate when using the height reference.

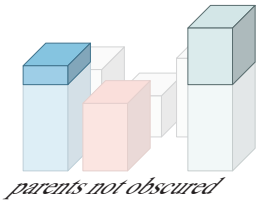
The study was conducted with 20 participants and the following preliminary results:

- H_1 (Speed) not confirmed: The average time to response for each task was slightly faster without the use of the height reference.
- H_2 (Accuracy) confirmed: Using the height reference increased the accuracy by 25% (similar nodes) to 42% (top 3 nodes) over the task completed without height reference.



Conclusion and Contact Information





This talk presented

- the concept of a height reference,
- an approach for its visual display, and
- a respective basic interaction model

in order to facilitate height-based queries in 2.5D treemaps.

Author email addresses:

- daniel.limberger@hpi.de
- matthias.trapp@hpi.de
- juergen-doellner@hpi.de

Bibliography I

[1] Bladh, T., Carr, D. A., and Scholl, J.
Extending tree-maps to three dimensions: A comparative study.
In *Proc. APCHI* (2004), pp. 50–59.

[2] Bohnet, J., and Döllner, J.
Monitoring code quality and development activity by software maps.
In *Proc. ACM MTD* (2011), pp. 9–16.

[3] Johnson, B., and Shneiderman, B.
Treemaps: A space-filling approach to the visualization of hierarchical information structures.
In *Proc. IEEE VIS* (1991), pp. 284–291.

[4] McCabe, T. J.
A complexity measure.
vol. SE-2, pp. 308–320.

Bibliography II

[5] Mitchell, W., Shook, D., and Shah, S. L.
A picture worth a thousand control loops: An innovative way of visualizing controller performance data.
In *Invited Plenary Presentation, Control Systems* (2004).

[6] Shneiderman, B.
Tree visualization with treemaps: A 2D space-filling approach.
ACM Trans. Graph. 11, 1 (1992), 92–99.

[7] Vliegen, R., van Wijk, J. J., and van der Linden, E.-J.
Visualizing business data with generalized treemaps.
IEEE Trans. Vis. Comput. Graph. 12, 5 (2006), 789–796.

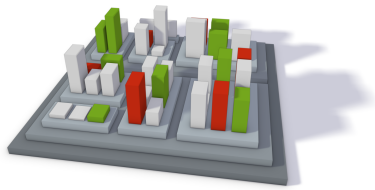
[8] Wattenberg, M.
Visualizing the stock market.
In *Proc. ACM CHI EA* (1999), pp. 188–189.

Bibliography III

[9] Wettel, R., and Lanza, M.
CodeCity: 3d visualization of large-scale software.
In *Proc. ACM ICSE Companion* (2008), pp. 921–922.

Software Maps — Map Theme

For a given task, a map theme specifies the mapping of attributes to visual variables.

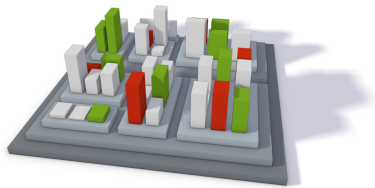


Technical Depth maps logic lines-of-code to weight, a nesting-level metric to color, and McCabe complexity [4]. It is used to reveal and monitor the 'technical debts' inherent to a software system's implementation.

Risk of Knowledge Drain maps logic lines-of-code to weight, the number of active developers to color, and a composite, nesting-level or McCabe based complexity measure indicating difficult-to-comprehend code to height. It is used to identify complex code units known only by few developers and reveal knowledge distribution.

Software Maps — Map Theme

For a given task, a map theme specifies the mapping of attributes to visual variables.

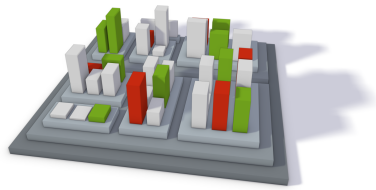


Technical Depth maps logic lines-of-code to weight, a nesting-level metric to color, and McCabe complexity [4]. It is used to reveal and monitor the 'technical debts' inherent to a software system's implementation.

Risk of Knowledge Drain maps logic lines-of-code to weight, the number of active developers to color, and a composite, nesting-level or McCabe based complexity measure indicating difficult-to-comprehend code to height. It is used to identify complex code units known only by few developers and reveal knowledge distribution.

Software Maps — Map Theme

For a given task, a map theme specifies the mapping of attributes to visual variables.



Technical Depth maps logic lines-of-code to weight, a nesting-level metric to color, and McCabe complexity [4]. It is used to reveal and monitor the 'technical debts' inherent to a software system's implementation.

Risk of Knowledge Drain maps logic lines-of-code to weight, the number of active developers to color, and a composite, nesting-level or McCabe based complexity measure indicating difficult-to-comprehend code to height. It is used to identify complex code units known only by few developers and reveal knowledge distribution.

Software Maps — Goals

Make code quality of software systems visible to “stakeholders in the development process, particularly, to the management” [2] by means of depicting, e.g., metrics and activity data.

Depending on the applied map theme Software Maps facilitate

- exploring structures,
- monitoring development processes,
- monitoring software quality, and
- identifying areas that require attention in the ongoing development process.